Prosit 2 – Machine Learning

De la donnée brute au Perceptron Multi-Couches

Prosit 2 – Machine Learning

Mars 2026

Table des matières

I Préparation des données **2**

1 Objectif **2**

2 Étape 1 – Imputation KNN **2**

3 Étape 2 – Encodage ordinal de `ocean_proximity` **2**

4 Étape 3 – Nettoyage des outliers **2**
4.1 Suppression des prix plafonnés 2
4.2 Suppression des outliers IQR 2

5 Étape 4 – Normalisation Z-Score **3**

6 Schéma du dataset final **3**

II Modélisation – Perceptron Multi-Couches **3**

7 Analyse de colinéarité **3**

8 Séparation Train / Test **4**

9 Architecture du MLP **4**

III Évaluation du modèle **4**

10 Métriques utilisées **4**
10.1 MAE – Mean Absolute Error 5
10.2 RMSE – Root Mean Squared Error 5
10.3 MAPE – Mean Absolute Percentage Error 5
10.4 R^2 – Coefficient de détermination 5

11 Résultats **5**
11.1 Avant nettoyage (20 640 lignes) 5
11.2 Après nettoyage (17 601 lignes) 5
11.3 Comparaison 5

Roadmap du projet

1. Donnees brutes

`datasets/housing.csv`

- 20 640 lignes, 10 colonnes — 207 valeurs manquantes, outliers, feature categorielle, echelles heterogenes

↓

2. Transformation — data_transform.ipynb

Etape	Methode	Resultat
Imputation	KNN (k=5, distance)	0 valeurs manquantes
Encodage	Ordinal (ocean_proximity)	Colonnes numeriques
Outliers	Suppression prix plafonnes + IQR	17 601 lignes
Normalisation	Z-Score (features uniquement)	Moyenne 0, σ 1

→ `datasets/amputed_normalized_housing_data.csv`

↓

3a. Modele MLP — predictive_perceptron.ipynb

- Colinearite → 5 features | Split 70/30 | Architecture : 5→64→32→1 (ReLU, Adam) | Eval : MAE, RMSE, MAPE, R^2
- Export : models/mlp_model.joblib + scaler.joblib

3b. Regression lineaire — predictive_linear_regression.ipynb

- Scatter plots → observation linearite | Feature : median_income | Split 70/30 | statsmodels OLS
- Eval : moyenne et ecart-type des erreurs (%), R^2

↓

4. Prediction interactive — predict.py

- Charge le modele MLP | Saisie 9 features | Normalisation Z-Score | Prix predit + marge (MAPE)

Première partie

Préparation des données

1 Objectif

Transformer le dataset brut `housing.csv` (20 640 lignes, 10 colonnes) en un fichier prêt pour l'entraînement d'un perceptron : `amputed_normalized_housing_data.csv`.

Pipeline : imputation KNN → encodage ordinal → nettoyage outliers → normalisation Z-Score.

2 Étape 1 – Imputation KNN

La colonne `total_bedrooms` contient 207 valeurs manquantes. On les comble par KNN ($k = 5$, pondération par distance inverse) :

$$\hat{x}_i = \frac{\sum_{j \in \mathcal{N}_k(i)} w_j \cdot x_j}{\sum_{j \in \mathcal{N}_k(i)} w_j}, \quad w_j = \frac{1}{d(i,j)} \tag{1}$$

Les données sont standardisées temporairement avant KNN pour que la distance euclidienne soit cohérente entre colonnes d'échelles différentes, puis dé-standardisées après imputation.

3 Étape 2 – Encodage ordinal de `ocean_proximity`

Catégorie	Valeur
`<1H OCEAN`	0
`INLAND`	1
`ISLAND`	2
`NEAR BAY`	3
`NEAR OCEAN`	4

Table 1 – Encodage ordinal de `ocean_proximity`

Cet encodage est réalisé **avant** l'imputation KNN (KNNImputer requiert des données numériques).

4 Étape 3 – Nettoyage des outliers

Deux types de nettoyage sont appliqués après imputation :

4.1 Suppression des prix plafonnés

965 lignes avec `median_house_value` = 500 001 \$ sont supprimées. Ces valeurs sont **censurées** (le prix réel est supérieur mais non enregistré), ce qui biaiserait l'apprentissage.

4.2 Suppression des outliers IQR

Pour chaque feature numérique (sauf `ocean_proximity` et `median_house_value`), on supprime les lignes dont la valeur est en dehors de l'intervalle :

$$[Q_1 - 1.5 \times IQR,\ Q_3 + 1.5 \times IQR] \tag{2}$$

où $IQR = Q_3 - Q_1$ est l'écart interquartile.

Feature	Outliers	Bornes
`total_rooms`	1 251	$[-1085, 5643]$
`total_bedrooms`	1 227	$[-230, 1175]$
`population`	1 126	$[-629, 3171]$
`households`	1 165	$[-204, 1092]$
`median_income`	364	$[-0.6, 7.7]$

Table 2 – Outliers détectés par feature (méthode IQR)

Bilan : 3 039 lignes supprimées (14.7%), **17 601 lignes restantes**.

5 Étape 4 – Normalisation Z-Score

Toutes les features sauf `median_house_value` (variable cible) sont normalisées :

$$z = \frac{x - \mu}{\sigma} \tag{3}$$

Justification : le perceptron calcule $\hat{y} = f(\mathbf{w}^T\mathbf{x} + b)$. Sans normalisation, les features à grande échelle (`total_rooms` ~ 2000) domineraient le gradient face aux petites (`median_income` ~ 4).

`median_house_value` n'est pas transformée car c'est la cible de la régression.

6 Schéma du dataset final

Colonne	Transformation
`longitude`	Z-Score
`latitude`	Z-Score
`housing_median_age`	Z-Score
`total_rooms`	Z-Score
`total_bedrooms`	KNN + Z-Score
`population`	Z-Score
`households`	Z-Score
`median_income`	Z-Score
`median_house_value`	**Non (cible)**
`ocean_proximity`	Ordinal + Z-Score

Table 3 – Schéma de `amputed_normalized_housing_data.csv` – 17 601 lignes, 10 colonnes

Deuxième partie
Modélisation – Perceptron Multi-Couches

7 Analyse de colinéarité

Avant l'entraînement, on supprime les features redondantes (corrélation $|r| > 0.8$). Pour chaque paire colinéaire, on conserve la feature la mieux corrélée avec la cible.

Feature A	Feature B	$\|r\|$	Supprimée
longitude	latitude	0.925	longitude
total_rooms	total_bedrooms	0.930	total_bedrooms
total_rooms	population	0.857	population
total_rooms	households	0.918	households
total_bedrooms	population	0.878	population
total_bedrooms	households	0.980	total_bedrooms
population	households	0.907	population

TABLE 4 – Paires colinéaires et features supprimées

Features conservées (5) : `latitude`, `housing_median_age`, `total_rooms`, `median_income`, `ocean_proximity`.

Features supprimées (4) : `longitude`, `total_bedrooms`, `population`, `households`.

8 Séparation Train / Test

Le dataset (17 601 lignes après nettoyage) est divisé aléatoirement (`random_state=42`) :

— **Train** : 12 320 échantillons (70%)

— **Test** : 5 281 échantillons (30%)

9 Architecture du MLP

Perceptron multi-couches à 2 couches cachées :

$$\mathbf{5}\ (\text{entrée}) \to \mathbf{64}\ (\text{ReLU}) \to \mathbf{32}\ (\text{ReLU}) \to \mathbf{1}\ (\text{sortie, identité})$$

Propagation avant :

$$h_1 = \text{ReLU}(W_1 x + b_1) \tag{4}$$
$$h_2 = \text{ReLU}(W_2 h_1 + b_2) \tag{5}$$
$$\hat{y} = W_3 h_2 + b_3 \tag{6}$$

Où $\text{ReLU}(z) = \max(0, z)$.

Hyperparamètres :

— Optimiseur : Adam (learning rate = 0.001)

— Loss : MSE (Mean Squared Error)

— Early stopping sur 10% de validation

— Max 500 epochs — convergé en **271 epochs**

— Nombre total de poids : **2 497**

Troisième partie
Évaluation du modèle

10 Métriques utilisées

C'est un problème de **régression**, pas de classification : il n'y a pas de « précision » (*accuracy*) au sens classique. On utilise 4 métriques complémentaires.

10.1 MAE – Mean Absolute Error

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{7}$$

Échelle : dollars ($). Erreur moyenne en valeur absolue — robuste aux outliers.

10.2 RMSE – Root Mean Squared Error

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{8}$$

Échelle : dollars ($). Pénalise les grosses erreurs plus fortement que la MAE (effet quadratique).

10.3 MAPE – Mean Absolute Percentage Error

$$\text{MAPE} = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \tag{9}$$

Échelle : pourcentage (%). Erreur relative — indépendante de l'échelle des prix. On peut l'interpréter comme : « en moyenne, le modèle se trompe de X% du prix réel ».

10.4 R^2 – Coefficient de détermination

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{10}$$

Échelle : sans unité, entre 0 et 1. Part de la variance du prix expliquée par le modèle. C'est la métrique la plus proche d'une « précision » pour la régression.

11 Résultats

11.1 Avant nettoyage (20 640 lignes)

	MAE ($)	**RMSE ($)**	**MAPE (%)**	**R^2**
Train (70%)	52 455	73 146	29.85	0.60
Test (30%)	52 183	72 690	29.97	0.60

Table 5 – Performance avant nettoyage des outliers

11.2 Après nettoyage (17 601 lignes)

11.3 Comparaison

12 Comment lire ces résultats ?

— **MAE et RMSE en baisse** : le modèle fait des erreurs plus petites en valeur absolue (−12% de RMSE). C'est l'amélioration la plus concrète.

	MAE ($)	**RMSE ($)**	**MAPE (%)**	**R^2**
Train (70%)	45 840	62 841	28.24	0.56
Test (30%)	46 288	63 958	27.98	0.55

TABLE 6 – Performance après nettoyage des outliers

Métrique (test)	**Avant**	**Après**	**Amélioration**
MAE ($)	52 183	46 288	−11.3%
RMSE ($)	72 690	63 958	−12.0%
MAPE (%)	29.97	27.98	−2.0 pts
R^2	0.60	0.55	−0.05

TABLE 7 – Impact du nettoyage des outliers sur la performance

— **MAPE en baisse (28%)** : en moyenne, la prédiction s'écarte de 28% du prix réel, soit une « précision relative » d'environ **72%**.
— **R^2 légèrement plus bas (0.55 vs 0.60)** : normal — en supprimant les outliers, on réduit la variance totale des prix. Le dénominateur de R^2 diminue, donc le ratio peut baisser même si les erreurs absolues sont plus petites. La MAE et le RMSE sont de meilleurs indicateurs ici.
— **Train ≈ Test** : pas de sur-apprentissage.

13 Limites identifiées

— 4 features supprimées par colinéarité — un modèle non-linéaire pourrait en tirer plus d'information.
— Seules 5 features utilisées — des données supplémentaires (surface, nombre de pièces, quartier) amélioreraient la prédiction.
— Le seuil IQR (1.5×) est un choix conservateur ; un seuil plus souple (2× ou 3×) garderait plus de données.

Partie III : Regression Lineaire

14. Objectif

Predire le prix median immobilier (**median_house_value**) a partir d'une seule feature : **median_income**. L'analyse des scatter plots (feature vs cible) montre que median_income presente la correlation lineaire la plus forte avec la variable cible.

15. Observation de la linearite

Avant de construire le modele, on visualise la relation entre chaque feature numerique et median_house_value via des scatter plots avec droite de regression et coefficient de correlation r. Cela permet de verifier visuellement si l'hypothese de linearite est raisonnable.

Resultat : **median_income** est la feature avec la correlation la plus forte et la tendance lineaire la plus nette. Les autres features montrent des relations faibles ou non lineaires.

16. Modele de regression lineaire

Le modele de regression lineaire simple apprend deux parametres (pente et intercept) qui minimisent l'erreur quadratique moyenne (MSE) :

$$y\blacksquare = \theta\blacksquare \cdot \text{median_income} + \theta\blacksquare$$

La solution analytique est donnee par l'equation normale :

$$\theta = (X\blacksquare X)\blacksquare^{1} X\blacksquare y$$

Implementation : **sklearn.linear_model.LinearRegression**, meme split 70/30 (random_state=42) que le modele MLP.

17. Evaluation — Moyenne et ecart-type des erreurs (%)

On evalue la precision du modele en analysant la distribution des **erreurs relatives** (residus en pourcentage) :

$$e(\%) = (y - y\blacksquare) / y \times 100$$

Les deux metriques principales :

Metrique	Interpretation
Moyenne des erreurs (%)	Biais relatif du modele. 0% = pas de biais systematique.
Ecart-type des erreurs (%)	Dispersion des erreurs. Plus bas = plus precis. Definit les intervalles de confiance a 1σ (68%) et 2σ (95%).

18. Coefficient de determination R²

$$R^2 = 1 - V(\text{reel}) / V(\text{theorique})$$

Variable	Description
$V(\text{reel}) = \Sigma(y\blacksquare - \blacksquare\blacksquare)^2$	Variance residuelle : erreur reelle du modele
$V(\text{theorique}) = \Sigma(y\blacksquare - \blacksquare)^2$	Variance totale : dispersion des donnees autour de la moyenne
y■	Valeur reelle (prix median observe)
■■	Valeur predite par le modele

■	Moyenne des valeurs reelles

- $R^2 = 1$: prediction parfaite (V(reel) = 0)
- $R^2 = 0$: le modele ne fait pas mieux que la moyenne
- $R^2 < 0$: le modele fait pire que la moyenne

19. Comparaison avec le MLP

Metrique	Regression lineaire	MLP (5→64→32→1)
Architecture	1 feature, 2 parametres	5 features, ~4000 parametres
Avantage	Simple, interpretable, rapide	Capture les relations non lineaires
Limite	Suppose une relation lineaire	Risque de surapprentissage, boite noire

Comparaison des modeles

Trois modeles ont ete entraines sur le meme dataset nettoye (17 601 lignes, split 70/30, random_state=42) pour predire le prix median immobilier :

1. Regression lineaire (statsmodels OLS)

Modele le plus simple : une droite ajustee par moindres carres. Utilise 5 features apres analyse de colinearite. Avantage : interpretable, rapide, fournit des p-values et intervalles de confiance. Limite : suppose une relation lineaire entre features et cible.

2. Random Forest (100 arbres)

Ensemble de 100 arbres de decision entraines sur des sous-echantillons bootstrap. Prediction = moyenne des arbres. 5 features. Avantage : capture les relations non lineaires, robuste au bruit. Limite : moins interpretable, risque de surapprentissage si non regularise.

3. Perceptron Multi-Couches (MLP)

Reseau de neurones a 2 couches cachees (5 -> 64 -> 32 -> 1), activation ReLU, optimiseur Adam. 5 features, early stopping sur validation. Avantage : capture des relations complexes non lineaires, architecture flexible. Limite : boite noire, necessite plus de donnees et de temps d'entrainement.

Tableau comparatif (jeu de test — 30%)

Metrique	Reg. lineaire	Random Forest	MLP
Features	5	5	5
Parametres	6	~100 arbres	~4 000
MAE ($)	53 956	~30 000	**~46 288**
RMSE ($)	71 641	~42 000	**~63 958**
MAPE (%)	~48	~18	**~28**
R^2	0.43	~0.80	**0.55**
Interpretabilite	Excellente	Moyenne	Faible
Risque surapprentissage	Faible	Eleve	Moyen

Modele retenu : MLP

Bien que le Random Forest affiche de meilleures metriques brutes sur le jeu de test, le **MLP est retenu** comme modele final pour les raisons suivantes :

• **Meilleure generalisation** : l'ecart train/test du Random Forest est tres eleve (R^2 train ~0.97 vs test ~0.80), signe de surapprentissage. Le MLP presente un ecart plus faible et donc une meilleure capacite de generalisation sur des donnees nouvelles.

• **Early stopping** : le MLP utilise un mecanisme d'arret precoce sur un jeu de validation (10% du train), ce qui regularise automatiquement le modele et limite le surapprentissage.

• **Deployabilite** : le modele MLP est exporte en joblib avec son scaler et ses metadonnees. Le script predict.py permet une prediction interactive en production.

• **Flexibilite** : l'architecture du MLP peut etre facilement ajustee (couches, neurones, learning rate) pour ameliorer les performances de facon iterative.

Glossaire

Prediction	Estimation d'une valeur future ou inconnue a partir d'un modele entraine sur des donnees historiques.
Traitement des donnees	Ensemble des operations appliquees aux donnees brutes (nettoyage, imputation, encodage, normalisation) pour les rendre exploitables par un algorithme de machine learning.
Modele de regression	Modele statistique qui apprend une relation entre des variables d'entree (features) et une variable de sortie continue (cible). Exemples : regression lineaire, perceptron multi-couches.
Surapprentissage / Sous-apprentissage	Surapprentissage (overfitting) : le modele memorise le bruit des donnees d'entrainement et generalise mal. Sous-apprentissage (underfitting) : le modele est trop simple pour capturer la structure des donnees.
ML (Machine Learning)	Branche de l'intelligence artificielle ou un algorithme apprend automatiquement des patterns a partir de donnees, sans etre explicitement programme pour chaque cas.
Coefficient de determination R^2	Metrique mesurant la part de variance de la cible expliquee par le modele. $R^2 = 1 - SS_{res} / SS_{tot}$. Vaut 1 pour un modele parfait, 0 si le modele ne fait pas mieux que la moyenne.
Erreur quadratique	Erreur calculee comme le carre de la difference entre valeur reelle et predite : $(y_i - ■_i)^2$. Penalise les grandes erreurs plus fortement. La MSE (Mean Squared Error) en est la moyenne.
Modeliser	Construire une representation mathematique d'un phenomene reel (ici, le prix immobilier) a l'aide d'un algorithme d'apprentissage et de donnees d'entrainement.
Parametre du modele	Valeurs internes apprises pendant l'entrainement (poids, biais). A distinguer des hyperparametres (learning rate, nombre de couches) fixes avant l'entrainement.
Regularisation	Technique ajoutant une penalite aux parametres du modele pour limiter le surapprentissage. Exemples : L1 (Lasso), L2 (Ridge), early stopping, dropout.
Industrialiser les processus	Deployer un modele de machine learning en production de maniere automatisee, reproductible et maintenable (pipelines CI/CD, monitoring, versionnement des modeles).
Prix median	Valeur centrale d'un ensemble de prix, telle que 50% des prix sont inferieurs et 50% sont superieurs. Plus robuste que la moyenne face aux valeurs extremes.
Entrainement	Phase ou le modele ajuste ses parametres en minimisant une fonction de cout sur les donnees d'entrainement (ex. descente de gradient sur la MSE).
Amelioration continue	Processus iteratif d'optimisation du modele : ajuster les hyperparametres, enrichir les donnees, tester de nouvelles architectures, et re-evaluer les performances.
Valeur manquante	Donnee absente dans le dataset (NaN). Doit etre traitee avant l'entrainement par imputation (KNN, moyenne, mediane) ou suppression de la ligne/colonne.
Donnees aberrantes (outliers)	Observations anormalement eloignees de la distribution principale. Detectees par la methode IQR ou le Z-Score, elles peuvent fausser l'apprentissage du modele.

Validation croisee	Methode d'evaluation qui divise les donnees en k sous-ensembles (folds). Le modele est entraine k fois, chaque fold servant une fois de jeu de test. Donne une estimation plus robuste de la performance.
Normaliser	Transformer les features pour qu'elles aient une echelle comparable. Z-Score : $z = (x - \mu) / \sigma$ (moyenne 0, ecart-type 1). Essentiel pour les algorithmes sensibles a l'echelle (perceptron, KNN).
Descente de gradient	Algorithme d'optimisation iteratif qui ajuste les parametres du modele dans la direction opposee au gradient de la fonction de cout : $\theta \leftarrow \theta - \alpha \cdot \nabla L(\theta)$, ou α est le learning rate.
Scikit-learn	Bibliotheque Python open-source pour le machine learning. Fournit des outils pour la preparation des donnees, l'entrainement de modeles et l'evaluation (LinearRegression, MLPRegressor, train_test_split, etc.).
Apprentissage supervise	Paradigme de ML ou le modele apprend a partir de paires (entree, sortie attendue). La sortie connue (label/cible) guide l'ajustement des parametres. Oppose a l'apprentissage non supervise (pas de labels).